Figure 28.1 Unencrypted HTTP requests can be read by users on the same network. Attackers can even intercept the request and response, reading or changing the data. HTTPS requests can't be read or manipulated by attackers.

Using unencrypted web apps in this way presents both a privacy and a security risk to your users. Attackers could read sensitive details such as passwords and personally identifiable information (PII), they could inject malicious code into your responses to attack users, or they could steal authentication cookies and impersonate the user on your app.

To protect your users, your app should encrypt the traffic between the user's browser and your app as it travels over the network by using the HTTPS protocol. This is similar to HTTP traffic, but it uses an SSL/TLS certificate to encrypt requests and responses, so attackers cannot read or modify the contents.

DEFINITION Secure Sockets Layer (SSL) is an older standard that facilitates HTTPS. The SSL protocol has been superseded by Transport Layer Security (TLS), so I'll be using TLS preferentially throughout this chapter. Normally, if you hear someone talking about SSL or SSL certificates, they actually mean TLS. You can find the RFC for the latest version of the TLS protocol at https://www.rfc-editor.org/rfc/rfc8446.

In browsers, you can tell that a site is using HTTPS by the https:// prefix to URLs (notice the s), or sometimes by a padlock, as shown in figure 28.2. Most modern browsers these days deemphasize that a site is using HTTPS, as most sites use HTTPS, and instead highlight when you're on a site that isn't using HTTPS, flagging it as insecure.